# Comparação entre RT-DETR-L e YOLO11m no WoodScape

Matheus Higino

15 de setembro de 2026

## 1. Objetivo

Este relatório consolida a primeira comparação experimental entre os modelos RT-DETR-L e YOLO11m para detecção de objetos em imagens fisheye do dataset WoodScape. O objetivo é avaliar não apenas a qualidade da detecção, mas também o custo computacional de cada arquitetura, considerando uma possível aplicação futura em sistemas de processamento em borda.

Os dois modelos foram treinados e avaliados utilizando a mesma divisão fixa do dataset, composta por conjuntos de treinamento, validação e teste. Foram consideradas cinco classes: `vehicles`, `person`, `bicycle`, `traffic_light` e `traffic_sign`.

## 2. Protocolo experimental

O dataset preparado contém 8.234 imagens, distribuídas da seguinte forma:

Tabela 1: Divisão utilizada nos experimentos.

| **Split** | **Imagens** | **Objetos** |
|---|---|---|
| Treinamento | 6.587 | 57.719 |
| Validação | 823 | 7.118 |
| Teste | 824 | 7.226 |
| Total | 8.234 | 72.063 |

As imagens relacionadas à mesma captura fisheye foram mantidas no mesmo *split*, reduzindo o risco de vazamento de informação entre treinamento, validação e teste.

Os dois experimentos utilizaram resolução de entrada 640, *batch size* 8, otimizador AdamW, *learning rate* inicial de 0,001, *weight decay* de 0,0005, *warm-up* de 3 épocas, *patience* de 20 épocas e *seed* 42.

Tabela 2: Configuração principal dos modelos.

| **Parâmetro** | **RT-DETR-L** | **YOLO11m** |
|---|---|---|
| Pesos iniciais | `rtdetr-l.pt` | `yolo11m.pt` |
| Épocas máximas | 100 | 100 |
| Melhor época | 71 | 71 |
| Early stopping | época 91 | época 91 |
| Batch size | 8 | 8 |
| Image size | 640 | 640 |
| Otimizador | AdamW | AdamW |
| Learning rate inicial | 0,001 | 0,001 |
| Weight decay | 0,0005 | 0,0005 |
| Warm-up | 3 épocas | 3 épocas |
| Patience | 20 | 20 |
| Seed | 42 | 42 |
| AMP | habilitado | habilitado |

Os treinamentos foram executados em uma NVIDIA GeForce RTX 5070, com aproximadamente 12 GB de VRAM, utilizando PyTorch 2.14.0+cu130, Python 3.14.4 e Ultralytics 8.4.147.

## 3. Resultados de validação

A Tabela a seguir apresenta os resultados obtidos pelos melhores checkpoints no conjunto de validação.

Tabela 3: Comparação no conjunto de validação.

| **Métrica** | **RT-DETR-L** | **YOLO11m** |
|---|---:|---:|
| Precision | 0,655 | **0,672** |
| Recall | **0,593** | 0,580 |
| mAP@50 | 0,618 | **0,620** |
| mAP@50:95 | 0,394 | **0,408** |
| Inferência (ms/imagem) | 5,8 | **2,7** |
| Parâmetros (milhões) | 31,99 | **20,03** |
| GFLOPs | 105,4 | **67,8** |
| Tamanho do modelo (MB) | 66,2 | **40,5** |

Na validação, o YOLO11m apresentou vantagem em Precision, mAP@50 e mAP@50:95, além de menor custo computacional. O RT-DETR-L apresentou Recall ligeiramente superior.

Esses resultados sugeriam inicialmente uma pequena vantagem global do YOLO11m, principalmente quando se considera conjuntamente desempenho de detecção e eficiência.

## 4. Resultados no conjunto de teste

Após a seleção dos melhores checkpoints exclusivamente a partir da validação, ambos os modelos foram avaliados no conjunto de teste, contendo 824 imagens e 7.226 instâncias.

Tabela 4: Resultados globais no conjunto de teste.

| Métrica | RT-DETR-L | YOLO11m |
|---|---|---|
| Precision | **0,6941** | 0,6800 |
| Recall | **0,5876** | 0,5604 |
| mAP@50 | **0,6270** | 0,6163 |
| mAP@50:95 | **0,4012** | 0,3976 |
| Inferência (ms/imagem) | 10,01 | **4,70** |
| Parâmetros (milhões) | 31,99 | **20,03** |
| GFLOPs | 105,4 | **67,8** |

No conjunto de teste, o RT-DETR-L apresentou os maiores valores globais de Precision, Recall, mAP@50 e mAP@50:95. Entretanto, a diferença em mAP@50:95 foi pequena: 0,4012 para o RT-DETR-L contra 0,3976 para o YOLO11m, uma diferença absoluta de 0,0036.

O resultado de teste também mostra uma inversão em relação ao conjunto de validação. Enquanto o YOLO11m havia apresentado mAP@50:95 superior na validação, o RT-DETR-L obteve uma pequena vantagem no teste. Essa diferença reforça a importância de manter um conjunto de teste independente para estimar a capacidade de generalização dos modelos.

## 5. Desempenho por classe no teste

Tabela 5: mAP por classe no conjunto de teste.

| | RT-DETR-L | | YOLO11m | |
|---|---|---|---|---|
| **Classe** | **mAP@50** | **mAP@50:95** | **mAP@50** | **mAP@50:95** |
| vehicles | 0,796 | 0,598 | **0,804** | **0,612** |
| person | 0,704 | 0,433 | **0,710** | **0,441** |
| bicycle | 0,504 | **0,317** | 0,504 | 0,309 |
| traffic_light | **0,574** | **0,301** | 0,522 | 0,288 |
| traffic_sign | **0,556** | **0,358** | 0,542 | 0,338 |

O YOLO11m apresentou os melhores valores de mAP@50:95 para as classes `vehicles` e `person`. Esse resultado é relevante para aplicações de detecção de obstáculos, nas quais veículos e pedestres representam classes de interesse prioritário.

O RT-DETR-L apresentou resultados superiores para `bicycle`, `traffic_light` e `traffic_sign`. A maior diferença entre os dois modelos ocorreu em `traffic_sign`, com vantagem de 0,020 em mAP@50:95 para o RT-DETR-L.

## 6. Eficiência computacional

A diferença mais evidente entre os modelos ocorreu no custo computacional.

O RT-DETR-L possui aproximadamente 31,99 milhões de parâmetros e requer 105,4 GFLOPs, enquanto o YOLO11m possui aproximadamente 20,03 milhões de parâmetros e requer 67,8 GFLOPs.

Em relação ao RT-DETR-L, o YOLO11m apresenta aproximadamente:

- 37,4% menos parâmetros;
- 35,7% menos GFLOPs;
- 53,1% menor latência de inferência no teste;

- velocidade de inferência aproximadamente 2,1 vezes maior nas condições avaliadas.

A latência medida no conjunto de teste foi de aproximadamente 10,01 ms por imagem para o RT-DETR-L e 4,70 ms por imagem para o YOLO11m.

Embora a medição de latência dependa do hardware, implementação, tamanho de lote e demais condições de execução, ambos os modelos foram avaliados no mesmo ambiente e pelo mesmo protocolo, permitindo uma comparação interna consistente.

## 7. Discussão

Os resultados indicam que não há um vencedor absoluto quando são considerados simultaneamente desempenho de detecção e eficiência computacional.

O RT-DETR-L apresentou o melhor desempenho global no conjunto de teste, porém com margens pequenas. A diferença de mAP@50:95 foi de apenas 0,0036, enquanto a diferença de mAP@50 foi de 0,0107.

Por outro lado, o YOLO11m alcançou desempenho muito próximo utilizando uma arquitetura substancialmente mais leve. Além da redução de parâmetros e GFLOPs, o modelo apresentou inferência aproximadamente duas vezes mais rápida.

Essa característica torna o YOLO11m especialmente relevante para cenários com restrições de processamento e memória, como aplicações em *edge computing*. Além disso, o YOLO11m apresentou melhor desempenho em mAP@50:95 nas classes `vehicles` e `person`, que possuem importância direta em cenários de detecção de obstáculos.

O comportamento observado entre validação e teste também merece destaque. O YOLO11m apresentou o maior mAP@50:95 na validação, enquanto o RT-DETR-L apresentou o maior valor no teste. Como as diferenças são pequenas, os resultados sugerem que ambas as arquiteturas apresentam desempenho competitivo no WoodScape e que conclusões baseadas exclusivamente no conjunto de validação poderiam superestimar pequenas diferenças entre os modelos.

## 8. Limitações

Os resultados apresentados correspondem a uma única divisão fixa do WoodScape e a uma única *seed* experimental. Portanto, pequenas diferenças entre os modelos não devem ser interpretadas automaticamente como evidência de superioridade estatística.

Também não foi realizada validação cruzada nem repetição dos treinamentos com múltiplas *seeds*. Além disso, a avaliação atual considera as métricas globais e por classe, mas ainda não separa explicitamente o desempenho de acordo com o tamanho dos objetos, posição no campo fisheye, nível de distorção ou condições visuais específicas.

Esses fatores serão importantes para aprofundar a análise dos modelos.

## 9. Próximas etapas

Como continuidade desta etapa experimental, são propostas as seguintes atividades:

1. analisar as curvas Precision-Recall, Precision, Recall e F1 dos dois modelos;
2. comparar as matrizes de confusão;
3. investigar falsos positivos e falsos negativos;
4. avaliar o desempenho em objetos pequenos, médios e grandes;
5. analisar objetos localizados nas regiões periféricas das imagens fisheye;

6. treinar uma variante mais leve, como YOLO11s, para ampliar a análise de eficiência;
7. investigar técnicas de *data augmentation* voltadas para robustez;
8. estudar exportação para ONNX/TensorRT e execução em dispositivos de borda.

## 10. Conclusão

A comparação entre RT-DETR-L e YOLO11m mostrou que ambas as arquiteturas apresentam desempenho competitivo para detecção de objetos em imagens fisheye do WoodScape.

O RT-DETR-L obteve o melhor desempenho global no conjunto de teste, com Precision de 0,6941, Recall de 0,5876, mAP@50 de 0,6270 e mAP@50:95 de 0,4012.

O YOLO11m obteve Precision de 0,6800, Recall de 0,5604, mAP@50 de 0,6163 e mAP@50:95 de 0,3976. Apesar da pequena redução nas métricas globais, apresentou vantagens expressivas de eficiência, com menor número de parâmetros, menor custo em GFLOPs e inferência aproximadamente 2,1 vezes mais rápida.

Dessa forma, o RT-DETR-L apresentou pequena vantagem em desempenho global, enquanto o YOLO11m demonstrou melhor relação entre desempenho e custo computacional. Para cenários futuros de processamento em borda, essa relação torna o YOLO11m um candidato particularmente relevante para investigações adicionais.

## Referências


[1] YOGAMANI, S. et al. **WoodScape: A Multi-Task, Multi-Camera Fisheye Dataset for Autonomous Driving**. In: IEEE/CVF International Conference on Computer Vision (ICCV), 2019.

[2] VALEO AI. **WoodScape: A multi-task, multi-camera fisheye dataset for autonomous driving**. Disponível em: `https://github.com/valeoai/WoodScape`. Acesso em: 14 set. 2026.

[3] ZHAO, Y.; LV, W.; XU, S.; WEI, J.; WANG, G.; DANG, Q.; LIU, Y.; CHEN, J. **DETRs Beat YOLOs on Real-time Object Detection**. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, p. 16965–16974.

[4] ULTRALYTICS. **RT-DETR Real-Time Detection Transformer**. Disponível em: `https://docs.ultralytics.com/models/rtdetr/`. Acesso em: 14 set. 2026.

[5] JOCHER, Glenn; QIU, Jing. **Ultralytics YOLO11**. Versão 11.0.0, 2024. Disponível em: `https://github.com/ultralytics/ultralytics`. Acesso em: 14 set. 2026.